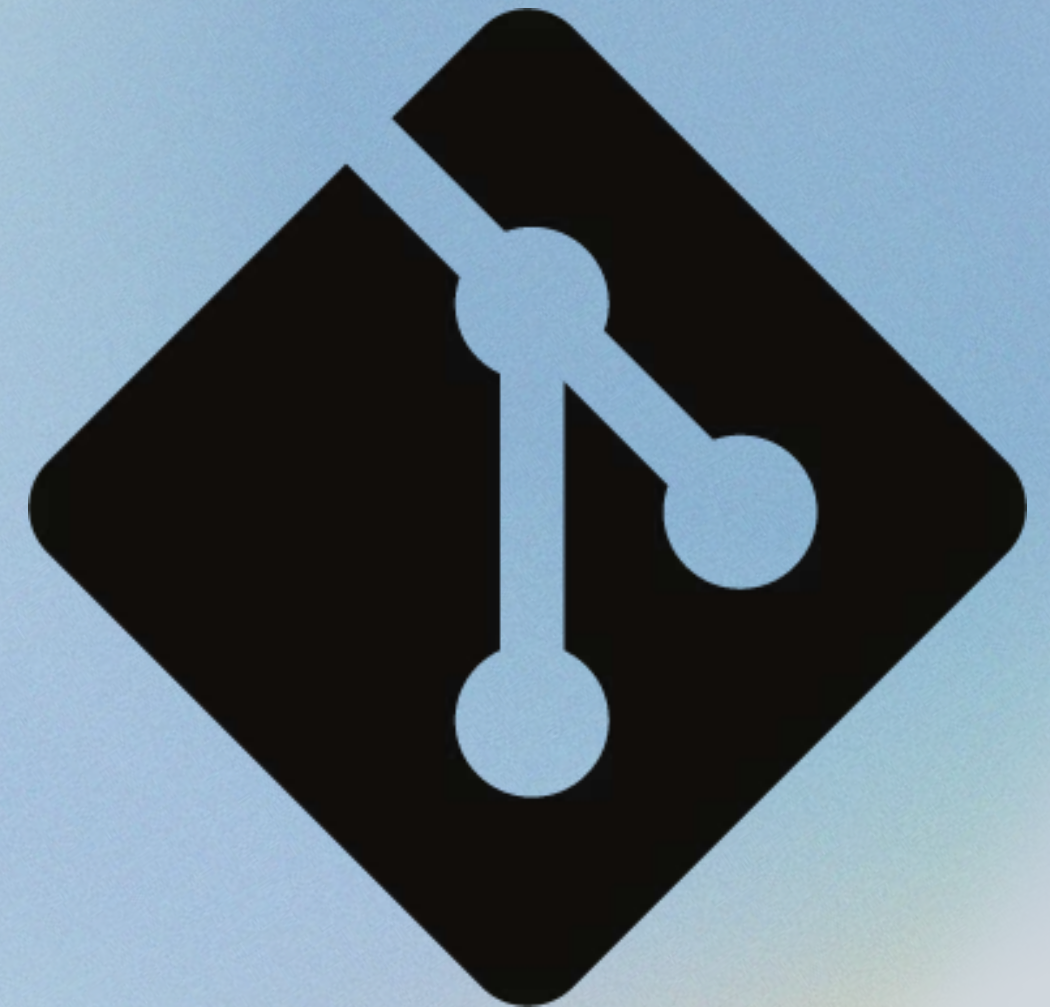




Fundamentals of Git & Github



Presented by:
Adam



Lesson Plan

- 01** Introduction to Git
- 02** Installation and setting up of Git
- 03** Initialising a repository
- 04** Committing in Git
- 05** Checking repo status & commits
- 06** Repository branches
- 07** Merging & deleting branches
- 08** Exploring old commits
- 09** Resolving merge conflicts
- 10** Activity: Upload to GitHub

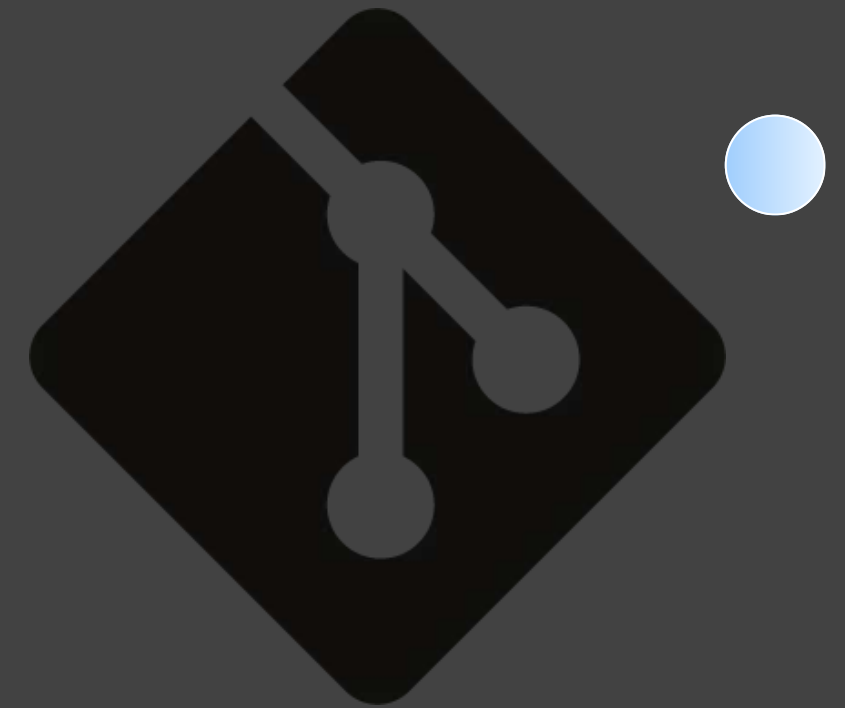
Introduction

Ever wanted to:

- Collaborate on the latest project version with your teammates?
- Retrieve a previously good version of a file?
- Work on different aspects of a project with your teammates simultaneously, without messing up the code?

We use Git to save our progress neatly, without having to save different versions of a file (*Project_Final*, *Project_Final_v2*, or *Project_Actual_Final*)

It is like a checkpoint in a video game, saving your work and allowing collaboration while being able to revert if something breaks.



Note:

1. Manual saving is required
2. Online services like GitHub is required to give others access to you project
3. Limited to tracking your project only



Activity: Setting up Git



Windows

- Download Git from: git-scm.com, install with default options
- Open a terminal (e.g. Git Bash)
- Run **git version** and check if the version is 2.55



MacOS

- Open Terminal
- Install Homebrew here: <https://brew.sh/>
- Run **brew install git** (*if you have not downloaded Git before*)
- Run **git version** and check if the version is 2.55

Note:

This workshop will be using Visual Studio Code as a default editor to use with Git
For participants with Git installed, please ensure that you are using version 2.55



Activity: Profile setup

Steps

Upon downloading, run: (2 dashes!)

```
git config --global user.name "Full Name"
```

```
git config --global user.email 'Email Address'
```

```
git config --global core.editor 'code --wait'
```

```
git config --global init.defaultBranch 'main'
```

Note

Set up your full name & email address

(saved to ~/.gitconfig file)

Set up your default editor (VSCode)

see this [link](#) for other editors

Initialising a default branch called “main”



Activity: Initialising a Repo

Steps

```
mkdir ~/Documents/sds_git
```

```
cd ~/Documents/sds_git
```

```
git init
```

```
git status
```

Note

Create a new empty directory

Change working directory

Initialising the Git repo

Check the directory is now a Git repo



Let's make your first commit

- Git tracks file changes through checkpoints known as “commits”.
- These commits are manually created by the user.
- Except for the first commit, it is built on top of another commit.
- Every commit is short and precise to include key information about the file changes.

What are the key information?

01

File contents

What was added, changed and deleted from a parent commit?

02

Commit message

What was changed and why?

03

Commit timestamp

When were the changes made?

04

Author's details

Who made the changes
(Name and email address)



Activity: Your first commit

Steps

In your code editor, open the folder and create a new file.

In your terminal, type stage your file as:

`git add < file 1 >`

or

`git add -A`

or

`git add .` to stage all files

Then, commit your staged files using:

`git commit -m 'commit message'`

Check your status using:

`git status`

If there is nothing left to commit, it should show:

“Nothing to commit, working tree clean.”

Amend your commit message using:

`git commit --amend -m 'amended commit message'`



Activity: Your first commit

Steps

You can also see your
history using:
git log

Commit hash



```
commit 86d1a4a5709247bbfe1be6dde0e05ddaae01d790 (HEAD -> main)
Author: <user><email>
Date: Sat Sep 19 00:40:43 2026 +0800
```

<message>

You can also view relationships
between commits using:
git log --graph

```
* commit 01a32d568071a0bbbb98e582b2095fe2f2e66f07 (HEAD -> main)
| Author: Adam Mikail <adammikail9999@gmail.com>
| Date: Sat Sep 19 01:24:51 2026 +0800
|
|   Chore: added mcspicy
|
* commit 86d1a4a5709247bbfe1be6dde0e05ddaae01d790
| Author: Adam Mikail <adammikail9999@gmail.com>
| Date: Sat Sep 19 00:40:43 2026 +0800
```

committing my first md file containing self-introduction

You can also view relationships in
one line using:
git log --oneline

```
adammikail@Adams-MacBook-Pro-2 SDS_git% git log --oneline
01a32d5 (HEAD -> main) Chore: added mcspicy
86d1a4a committing my first md file containing self-introduction
```



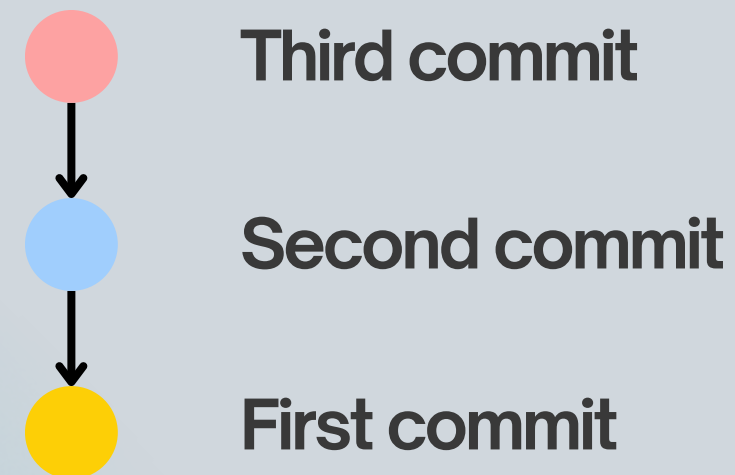
Let's Recap:

So far, what we have learnt:

- Initialising a Git repo
- Making a commit

Commands:

- `git init`
- `git status`
- `git add`
- `git commit`
- `git log`
- `git log --oneline`
- `git log --graph`



What if multiple people are:

- Working on the same project?
- Or on different features?
- Or your supervisor wants to review your code before rolling out easily?



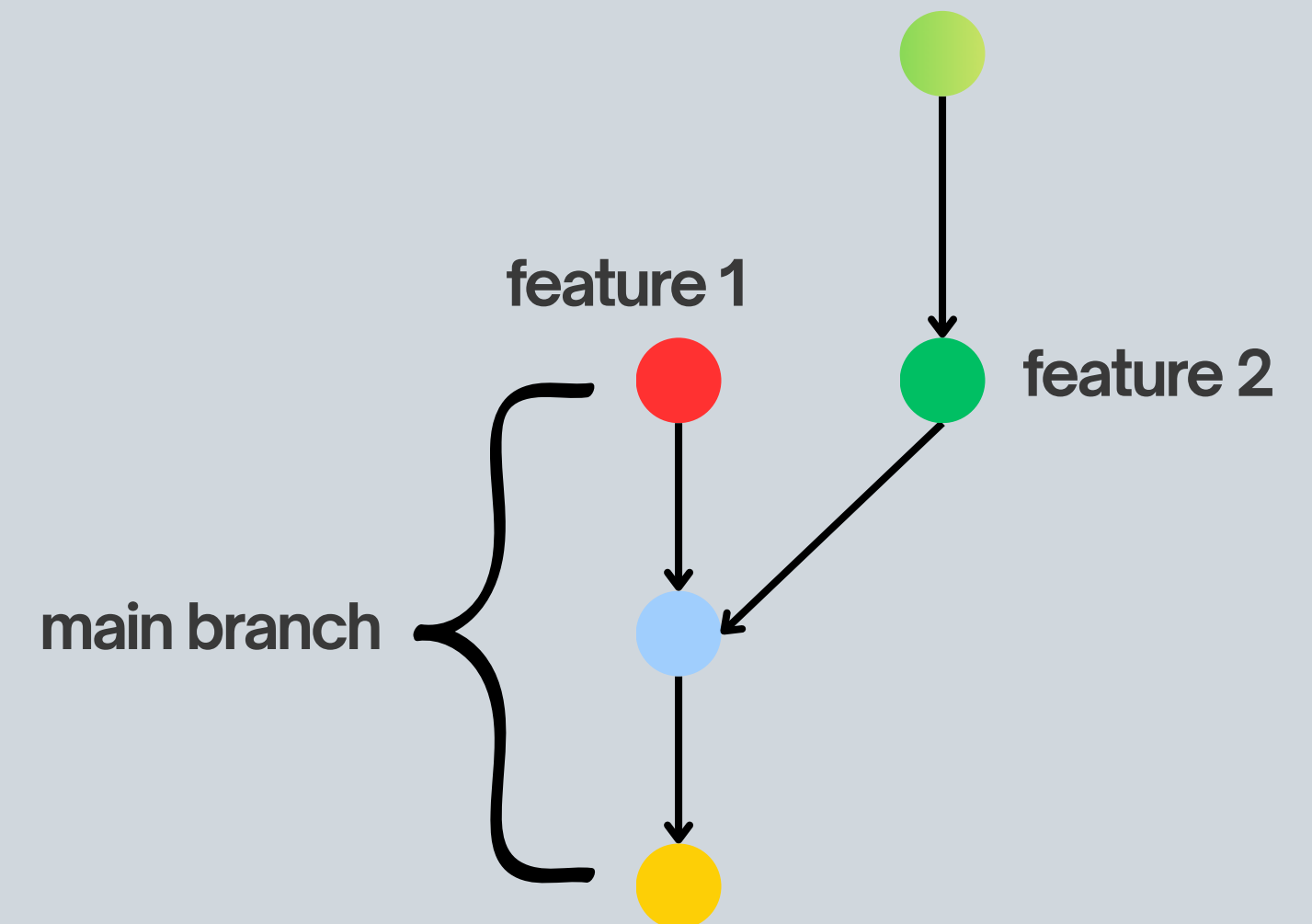
Branches

Purpose:

- Isolates your work by letting your main branch stay deployable.
- You experiment on a separate branch.
- On Github, this branch gets vetted by your supervisor, who then approves your work to the main branch.

Branch names can also be assigned in Git.

The initial branch name is “**master**” but we have assigned it to “**main**” in the profile setup.





Activity: Your first branch

Steps

You can see your active branches using:
`git branch`

You can also create new branches using:
`git branch <branch-name>`

- This branch points to the same commit as the old branch with the most recent commit
- Changing this branch does not change the main branch

You can also switch between branches using:
`git switch <branch-name>`

- This sets your <branch-name> as your current branch
- `git switch -c <branch-name>` creates a new branch and switches it at once



Activity: Your first branch

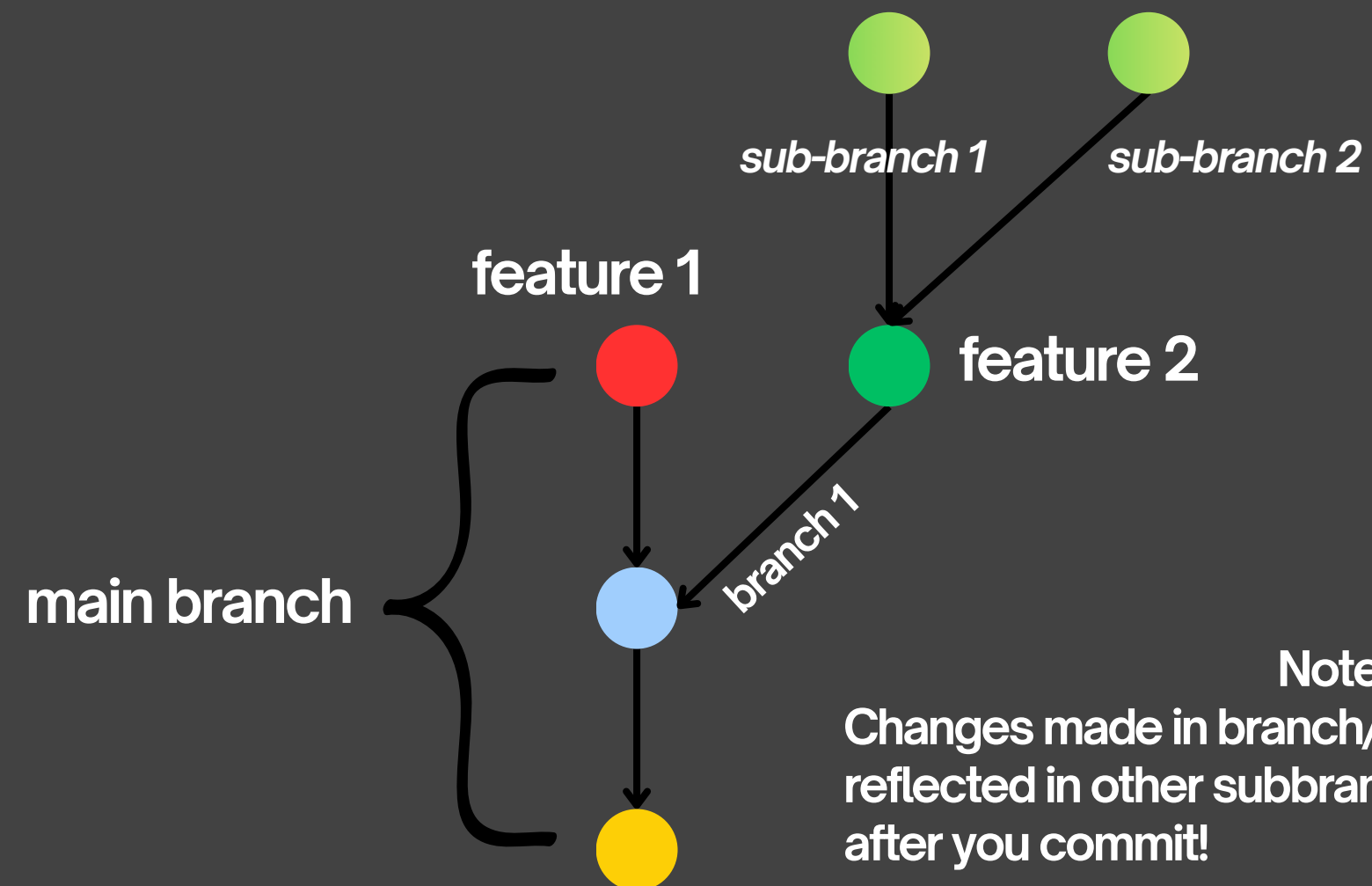
Steps

You can also make sub-branches under a branch

main → *branch-1* → *branch-1a*

`git branch <branch name>`

- This sets your <branch-name> as your current sub-branch
- There is no command to see a graph of all sub-branches. However using:
`git log --oneline --graph --all`
allows you to see the relationships



Note:

Changes made in branch/subbranch will not be reflected in other subbranch or branches even after you commit!



Merges

Once we are done with the assigned task, and the branch is no longer needed, we can merge the source branch with its destination branch!

Fast-forward merge

- Default merge if not specified
 - Main hasn't changed & you don't care about preserving "this was a separate feature" in history
- (Does not preserve history)*

No fast-forward merge

- Main hasn't changed, but you want the history to clearly show a feature branch existed
- (Preserves history)*

Squash merge

- Combines the entire branch's commits into a single new commit instead of preserving individual commits
- (Does not preserve history)*



Merges

Once we are done with the assigned task, and the branch is no longer needed, we can merge the source branch with its destination branch!

Scenario: Branching from main main: 6cc2491 sub-branch 1: 6ca3094 — 48296bf		
Fast-forward merge main: 6cc2491 — 6ca3094 — 48296bf Individual commits are preserved	No fast-forward merge main: 6cc2491 — M sub-branch 1: 6ca3094 — 48296bf Individual commits are preserved, merge commit created	Squash merge main: 6cc2491 — d1ab777 sub-branch 1: 6ca3094 — 48296bf Individual commits “squashed” into 1 new commit, no merge commit created



Activity: Merging Branches

Steps

To merge a source branch into the destination branch, we must be in the destination branch first

Fast-forward merge:

```
git merge <source-branch>
```

or

```
git merge --ff <source-branch>
```

No fast-forward merge:

```
git merge --no-ff <source-branch>
```

Squash merge:

```
git merge --squash <source-branch>
```

After merging, the destination branch will be updated with the new commits.

The source branch will not be modified in any way regardless of merge resolution. You can observe the changes using:

```
<git log --oneline --graph --all>
```



Activity: More with Branches

● Steps

Once we do not need the branch anymore, delete the branch using:

```
git branch -d <branch-name>
```

We can explore how the code looks like in old commits using:

```
git log (obtain target commit hash)
```

```
git switch -d <commit-hash>
```

However, if we run `git status`, we can see “HEAD detached at <commit-hash>”

HEAD points to where (in this case which branch/commit you are on.

```
git switch <existing-branch>  
(Returns back to existing branch)
```



Activity: Merge Conflicts

● Steps

When Git cannot automatically merge similar changes made by multiple commits on a same region, a merge conflict occurs!

Merge conflicts allow us to decide which line(s) of code do we want to keep

We can open the file contents, using any code editor. Git will add conflict markers to show the conflict.

Edit all the conflicting files and ensure that they are exactly the content you want to keep.

`git add -A`
(Stage all modified files)
then

`git merge --continue`
(Continues merge process)
or

`git merge --abort`
(Returns to the original repo)



Standard workflow

Notes

Even if you can make many branches, you should keep your branches organised!

- Keep “Main” branch as your production branch where the good working code is deployed.
- Keep branches short-lived, with one fixed task for clarity.
- Branch off the latest “Main” branch and not off another feature branch.
- Once your task is done, merge your sub-branch to branch, and sub-branch to main.
- Delete branches if they are no longer in use.



Let's push your first document

We will be using Github, which is an online service that allows collaboration using Git, storing your Git repositories online.

Some commands and terminologies to understand:

git clone <URL>

Downloads a full copy of a remote repository (including all its history) onto your computer.

git pull <remote-name><branch-name>

Downloads new changes from the remote and merges them into your current local branch.

git push <remote-name><branch-name>

Uploads your local commits to the remote repository.

git checkout -b <branch-name>

creates a new branch and switch to it

Pull Requests (PRs)

What it is: a request to merge your branch's changes into another branch (usually main), done through GitHub's website, not the terminal.



Typical workflow

Full life cycle of a project cycle:

clone → branch → add/commit → push → open PR → review → merge

↓

(get the repo) (pull, to stay updated)

From Branch to Merge Summary

1. `git checkout -b <branch-name>` (make a branch)
2. Make changes, git add, git commit
3. `git push -u origin <branch-name>` (upload your branch)
4. Go to GitHub → click "Compare & pull request"
5. Add a title/description → Create pull request
6. A reviewer checks it → approves → Merge
7. Your changes are now part of main, visible to everyone



Activity: Setting up Github

(RA)

Steps

Open <https://github.com/NUS-SDS-Workshops-AY2627/Sem-1>

Click **Fork** (top right, next to Star/Watch)

On the next screen, keep the default name (Sem-1) and owner (your account), then click **Create fork**

GitHub creates your own copy at <https://github.com/<your-username>/Sem-1>

clone your fork, not the original:

```
git clone https://github.com/<your-username>/Sem-1.git
```

```
cd Sem-1
```



Activity: Setting up Github

(WS)

Steps

Clone the online repo on Github to our computer,

```
git clone  
https://github.com/NUS-SDS-Workshops-AY2627/Sem-1.git
```

```
cd Sem-1
```

```
git checkout -b <your-name>  
intro
```

Copy the .md file you already have into the Git-Workshop folder inside this cloned repo.

```
cp /Desktop/<your-file>.md Git-Workshop/<your-name>.md
```

```
git add Git-Workshop/<your-name>.md
```

```
git commit -m "Add <your-name>  
introduction"
```

Push your branch upstream

(sending your local branch's commits up to GitHub)

```
git push -u origin <your-name>-intro
```

GitHub will give you a link after pushing, or go to the repo and click *Compare & pull request*.

Set the base branch to 'main', set myself (Adam210503) or Jason (JasonOng0109) as a reviewer, then submit your PR.



Statistics and Data Science Society

Presented by :
Adam Mikail

Thank You